

Data Analysis with Python 2024–2025

Parte 6: Machine Learning Types

Classificação Naive Bayes (Teoria)

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**.

Conteúdo

1 Machine Learning: Classificação Naive Bayes

A classificação é uma forma de aprendizado supervisionado (*supervised learning*). O objetivo é anotar todos os pontos de dados com um rótulo (*label*). Aqueles pontos que têm o mesmo rótulo pertencem à mesma classe. Pode haver dois ou mais rótulos. Por exemplo, uma forma de vida pode ser classificada (de forma ampla) com os rótulos: animal, planta, fungo, arquea, bactéria, protozoário e cromista. Os pontos de dados possuem certas características (*features*) observadas que podem ser usadas para prever seus rótulos. Por exemplo, se tem penas, então é mais provável que seja um animal.

No aprendizado supervisionado, um algoritmo recebe primeiro um conjunto de treinamento de dados com suas características e rótulos. Em seguida, o algoritmo aprende a partir dessas características e rótulos um modelo (probabilístico), que pode ser usado posteriormente para prever os rótulos de dados nunca antes vistos.

A classificação *Naive Bayes* (Bayes Ingênuo) é um método de classificação rápido e simples de entender. Sua velocidade se deve a algumas simplificações que fazemos sobre as distribuições de probabilidade subjacentes, a saber, a suposição sobre a independência das características. Ainda assim, pode ser bastante poderoso, especialmente quando há características suficientes nos dados.

Suponha que temos para cada rótulo L uma distribuição de probabilidade. Essa distribuição dá a probabilidade para cada combinação possível de características (um vetor de características): $P(\text{features} | L)$.

A ideia principal na classificação Bayesiana é inverter a direção da dependência: queremos prever o rótulo com base nas características: $P(L | \text{features})$. Isso é possível pelo teorema de Bayes:

$$P(L | \text{features}) = \frac{P(\text{features} | L)P(L)}{P(\text{features})}$$

Vamos assumir que temos dois rótulos L_1 e L_2 , e suas distribuições associadas: $P(\text{features} | L_1)$ e $P(\text{features} | L_2)$. Se temos um ponto de dados com certas "features", cujo rótulo não conhecemos, podemos tentar prevê-lo usando a razão das probabilidades a posteriori:

$$\frac{P(L_1 | \text{features})}{P(L_2 | \text{features})} = \frac{P(\text{features} | L_1)P(L_1)}{P(\text{features} | L_2)P(L_2)}$$

Se a razão for maior que um, rotulamos nosso ponto de dados com o rótulo L_1 , e se não, damos a ele o rótulo L_2 . As probabilidades a priori (*prior*) $P(L_1)$ e $P(L_2)$ dos rótulos podem ser facilmente descobertas a partir dos dados de entrada, pois para cada ponto de dados também temos o seu rótulo. O mesmo vale para as probabilidades de características condicionadas ao rótulo.

Demonstraremos primeiro a classificação Naive Bayes usando distribuições Gaussianas.

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import make_blobs

X, y = make_blobs(100, 2, centers=2, random_state=2, cluster_std=1.5)
colors = np.array(["red", "blue"])
plt.scatter(X[:, 0], X[:, 1], c=colors[y], s=50)
for label, c in enumerate(colors):
    plt.scatter([], [], c=c, label=str(label))
plt.legend();
```

O algoritmo Naive Bayes ajustou duas distribuições Gaussianas bidimensionais aos dados. As médias e as variâncias definem essas distribuições completamente.

```
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X, y);
print("Means:\n", model.theta_)
print("Variances:\n", model.var_)
```

Saída esperada:

```
Means: [[-1.64939095 -9.36891451]
        [ 1.29327924 -1.24101221]]
Variances: [[ 2.06097005  2.47716872]
            [ 3.33164807  2.22401384]]
```

1.1 Avaliando a precisão (Accuracy)

O *accuracy score* (pontuação de precisão) dá uma medida sobre quão bem conseguimos prever os rótulos. O valor máximo é 1.0.

```
from sklearn.metrics import accuracy_score

y_fitted = model.predict(X)
acc = accuracy_score(y, y_fitted)
print("Accuracy score is", acc)
# Accuracy score is 1.0
```

A pontuação foi a melhor possível, o que não é uma surpresa, já que tentamos prever os dados que já tínhamos visto! Mais tarde, dividiremos nossos dados em duas partes: uma para aprender o modelo e outra para testar suas habilidades preditivas.

1.2 Limitações e Dados Multivariados

O problema com a classificação Bayesiana "Ingênua" é que ela tenta modelar os dados usando distribuições Gaussianas alinhadas ao longo dos eixos X e Y (assumindo independência). Se tivermos dados "inclinados", como distribuições normais multivariadas com covariância cruzada, a classificação Naive Bayes perde parte da sua representação ideal porque pressupõe falsamente que as *features* não têm correlação.

Mesmo obtendo uma pontuação alta de precisão, ao desenhar os eixos podemos observar graficamente que o modelo não é perfeito para distribuições correlacionadas, precisando gerar elipses estritamente alinhadas aos eixos para estimar a classe.

2 Classificação de Textos

A seguir, tentaremos classificar um conjunto de mensagens postadas em um fórum público. As mensagens foram divididas em grupos por tópicos. Portanto, temos um conjunto de dados pronto para testes de classificação. Primeiro, vamos carregar esses dados usando o `scikit-learn` e imprimir as categorias de mensagens.

```
from sklearn.datasets import fetch_20newsgroups
data = fetch_20newsgroups()
print(data.target_names)
```

Focaremos em apenas quatro categorias de mensagens. A ferramenta `fetch_20newsgroups` nos permite dividir facilmente os dados em dados de treinamento e de teste.

```
categories = ['comp.graphics', 'rec.autos', 'sci.electronics', 'sci.
    crypt']
train = fetch_20newsgroups(subset='train', categories=categories)
test = fetch_20newsgroups(subset='test', categories=categories)

print("Training data:", len(train.data))
print("Test data:", len(test.data))
```

Usamos como características as frequências de cada palavra no conjunto de dados. Ou seja, existem tantas características quanto palavras distintas no conjunto de dados. Como as características agora são contagens, faz sentido usar a distribuição *multinomial* em vez da Gaussiana.

Vamos tentar modelar essas mensagens usando distribuições multinomiais. Cada categoria de mensagem tem sua própria distribuição. Uma distribuição multinomial possui f parâmetros não negativos $\theta_1, \dots, \theta_f$, que somam um. Por exemplo, o parâmetro θ_3 pode indicar a probabilidade da palavra "placa" aparecer em uma mensagem da categoria que essa distribuição está descrevendo.

No `scikit-learn`, existe uma classe `CountVectorizer` que converte mensagens em formato de strings de texto para vetores de características. Podemos integrar essa conversão com o modelo que estamos usando (Multinomial Naive Bayes), de modo que a conversão ocorra automaticamente como parte do método `fit`. Alcançamos essa integração usando a ferramenta `make_pipeline`.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

model = make_pipeline(CountVectorizer(), MultinomialNB())
model.fit(train.data, train.target)
labels_fitted = model.predict(test.data)
print("Accuracy score is", accuracy_score(labels_fitted, test.target))
# Accuracy score is ~ 0.9205
```

O classificador parece funcionar muito bem! Note que agora usamos dados separados para testar o modelo.

A matriz de características é armazenada em formato esparsa (*sparse format*), ou seja, apenas as contagens diferentes de zero são armazenadas para economizar memória e poder de processamento.